Super Resolution Using the **Diffusion Transformer (DiT)**

Comparing Diffusion Based and Direct Prediction Approaches for 64x64 to 128x128 Image Upscaling

Jianyu Jia

March 2026

1. Introduction

Image super resolution is the task of reconstructing a high-resolution image from a low-resolution input. It has practical applications in medical imaging, satellite imagery, surveillance footage enhancement, and consumer photo upscaling. This project explores how the Diffusion Transformer architecture, originally built by Facebook Research for generating images from class labels, can be remodeled for super resolution on a small agricultural dataset.

The core problem that super resolution solves is recovering detail that was lost when an image was captured at low quality or downsampled to a smaller size. A 64x64 image only has 4,096 pixels to represent an entire scene, so fine details like strawberry seeds, leaf veins, and surface textures are all compressed into a blur. A 128x128 image has four times as many pixels, giving enough room to represent those details. Simply stretching the small image to the larger size just produces a bigger blur. The model must intelligently guess what the missing detail should look like, based on patterns it learned during training from thousands of examples of what sharp images look like.

The DiT architecture was published by Peebles and Xie in 2022 as part of their work on scalable diffusion models. Their key contribution was replacing the U-Net backbone that most diffusion models relied on with a Vision Transformer, and introducing a conditioning mechanism called adaLN-Zero that lets external signals control how each transformer block processes data. The architecture achieved state-of-the-art results on ImageNet generation benchmarks.

This project takes the DiT architecture and applies it to a different problem: taking a 64x64 strawberry image and producing a 128x128 version. Two approaches were implemented and compared. The first uses the full diffusion process where the model learns to iteratively denoise images over many steps. The second removes the diffusion process entirely and has the model predict the high-resolution output directly in a single forward pass. Both approaches also compare training with and without data augmentation to measure its effect on generalization.

2. Architecture

Before DiT, most diffusion models used a U-Net as their backbone. A U-Net is a CNN-based architecture shaped like a U: it progressively downsamples the image through convolutional layers (from 128x128 down to 16x16), processes it at the small resolution, then upsamples back to the original size with skip connections carrying fine detail from the downsampling path to the upsampling path. Each convolutional layer only sees a small local window of pixels (typically 3x3), so the model builds up its understanding of the image gradually through many stacked layers.

DiT replaces this entire U-shaped CNN with a Vision Transformer. Instead of processing through convolutional layers, DiT chops the image into small patches, treats each patch as a token (the same way

a language model treats words), and runs them all through transformer blocks with self-attention. The fundamental difference is that self-attention lets every patch directly reference every other patch in a single layer. A patch in the top-left corner of the image can attend to a patch in the bottom-right corner immediately, which is something U-Net can only achieve after many layers of downsampling bring those distant regions close together in the feature maps. This global context is particularly useful for super-resolution because texture patterns like strawberry seeds repeat across the whole image, and the model benefits from seeing all of them at once.

Both models in this project share the same DiT transformer blocks extracted from Facebook's repository. The components taken directly from the original code include the DiTBlock with adaLN-Zero conditioning, the modulate function that applies learned shift and scale values to normalized features, sinusoidal 2D positional embeddings adapted from Facebook's MAE repository, and the zero-initialization strategy where gate values in each block start at zero so the block initially acts as an identity function.

The two models differ in how they use these building blocks to perform super-resolution.

2.1 Approach A: Diffusion-Based Model

The diffusion-based model preserves the iterative denoising process that DiT was originally designed for. It works by taking a noisy 128x128 image as input to the transformer, with the 64x64 low-resolution image encoded separately through a small CNN (called the LowResEncoder) into a conditioning vector. This conditioning vector is combined with a timestep embedding and fed into every DiT block through the adaLN-Zero mechanism. The model learns to predict the noise that was added at each timestep, and at inference time it starts from pure noise and gradually denoises over 50 steps to produce the output.

This model uses the DiT-S configuration with 12 transformer blocks, a hidden dimension of 384, and 6 attention heads, totaling approximately 34.3 million parameters. The noisy 128x128 image is split into 1024 patch tokens using a patch size of 4. A cosine noise schedule with 1000 timesteps was used, following the approach from the Improved DDPM paper by Nichol and Dhariwal (2021). The loss function is MSE between the predicted noise and the actual noise added during training.

2.2 Approach B: Direct Prediction Model

The direct prediction model strips out the diffusion process and instead takes the 64x64 image directly as input to the transformer. There is no noise, no timesteps, and no iterative denoising. The transformer processes the low-resolution patches, reconstructs features at 64x64 resolution, and then a PixelShuffle layer upsamples the result to 128x128 in one step.

PixelShuffle was chosen over transposed convolutions for the upsampling step because transposed convolutions are known to produce checkerboard artifacts in their output, while PixelShuffle rearranges feature channels into spatial dimensions cleanly.

This model uses a smaller configuration with 6 transformer blocks, a hidden dimension of 256, and 8 attention heads, totaling approximately 7.2 million parameters. Because the input is 64x64 instead of 128x128, the transformer only processes 256 patch tokens instead of 1024, which makes each training step significantly faster. The conditioning signal for adaLN-Zero is a single learned vector rather than an encoded image, since the low-resolution information is already in the patch tokens themselves. The loss function is L1 (mean absolute error), which was chosen over MSE because L1 penalizes large errors less aggressively and tends to produce sharper outputs rather than the blurry averages that MSE can encourage.

2.3 Key Differences

The diffusion model is larger at 34.3 million parameters versus 7.2 million for the direct prediction model. It processes a noisy 128x128 image through 1024 patch tokens and conditions on both a CNN-encoded low-resolution image and a timestep embedding, producing output through 50 iterative denoising steps with MSE loss on predicted noise. The direct prediction model takes the 64x64 image directly through 256 patch tokens, outputs 128x128 in a single forward pass via PixelShuffle, and trains with L1 loss on the predicted image. The smaller token count is the main reason the direct prediction model trained at roughly 8 seconds per epoch compared to 47 seconds for the diffusion model.

3. Dataset and Preprocessing

The dataset used is the Multi-Class Strawberry Ripeness Detection Dataset from Kaggle, containing 566 images of strawberries photographed at various ripeness stages in greenhouse environments. The dataset was originally designed for object detection with YOLO-format bounding box annotations, but since super-resolution is purely about pixel reconstruction, the class labels and bounding boxes were not used.

For the diffusion model, images were resized to 128x128 for the high-resolution target and 64x64 for the low-resolution input. Pixel values were normalized to the range negative one to one, which is standard for diffusion models because the noise addition process is centered at zero. The data was split 80/10/10 into training, validation, and test sets.

For the direct prediction model, the same image pairs were created but pixel values stayed in the zero to one range since L1 loss works well without centering. The split was 80/20 into training and test sets.

3.1 Data Augmentation

Both models were trained twice: once with augmentation and once without, to measure the effect on generalization. The augmentation transforms included random horizontal flips, random vertical flips, and random rotations at 90 degree increments. For the diffusion model, spatial transforms were applied identically to both the LR and HR images to maintain alignment, and color jitter was applied to the LR input only. For the direct prediction model, augmentation was applied to the full image before the LR/HR split, which automatically maintains alignment.

4. Training

Both models used the AdamW optimizer with a learning rate of 1e-4 and no weight decay, matching the hyperparameters from the original DiT paper. The diffusion model was trained for 100 epochs with gradient clipping at a max norm of 1.0 and a batch size of 8. The direct prediction model was trained for 100 epochs with a batch size of 32. Both ran on an NVIDIA Tesla T4 GPU through Kaggle.

4.1 How Diffusion Training Works

The diffusion model's training process is indirect. For each training step, the model takes a clean 128x128 image and adds a random amount of noise to it. The amount depends on which timestep is randomly sampled from the 1000 possible noise levels. A low timestep means very little noise (the image is almost clean), while a high timestep means heavy noise (the image is almost pure static). The model receives this noisy image along with the 64x64 low-resolution reference and the timestep number, and its job is to predict what noise was added. Not the clean image, just the noise. The loss is computed as the MSE between the predicted noise and the actual noise.

The model never directly sees or tries to produce the final output image during training. It only learns to identify noise patterns across all 1000 noise levels. This means the training signal is indirect: the model is getting feedback on noise prediction accuracy, not on image quality. At inference time, the model chains together 50 of these noise predictions, starting from pure static and removing a bit of noise at each step, to build the final output. If any of those 50 predictions are inaccurate because the model did not learn that particular noise level well enough, the error carries forward through every subsequent step and the output degrades. This compounding effect is why the diffusion model's visual outputs appeared noisy.

4.2 How Direct Prediction Training Works

The direct prediction model's training is straightforward. The model takes a 64x64 image, produces a 128x128 output in a single forward pass, and the loss is computed by directly comparing the output to the real 128x128 ground truth using L1 loss. Every training step gives the model clear, direct feedback: here is what you produced, here is what it should look like, minimize the difference. There is no noise, no

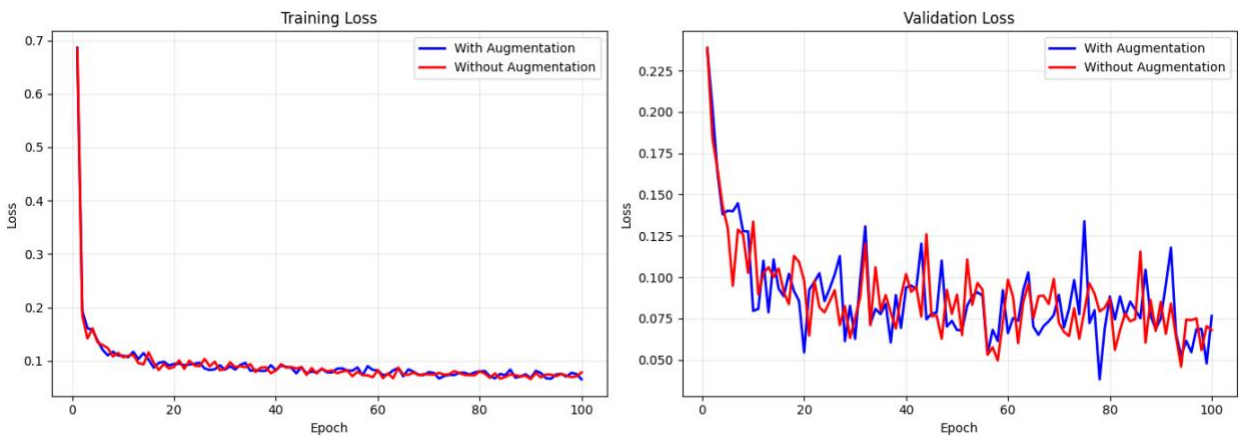
timesteps, and no multi-step process. The gradient signal is simple and unambiguous, which is why this approach converges faster and more reliably, especially on a small dataset.

For the diffusion model, the best checkpoint was saved based on the lowest validation loss during training. For the direct prediction model, PSNR and SSIM were computed on the test set after every epoch to track quality improvement over time.

5. Results

5.1 Diffusion Model Results

The diffusion model trained for 100 epochs with training loss decreasing from approximately 0.69 to 0.07. Validation loss showed high variance across epochs, which is characteristic of diffusion training where each pass samples random timesteps. The augmented model achieved slightly better results across all metrics.

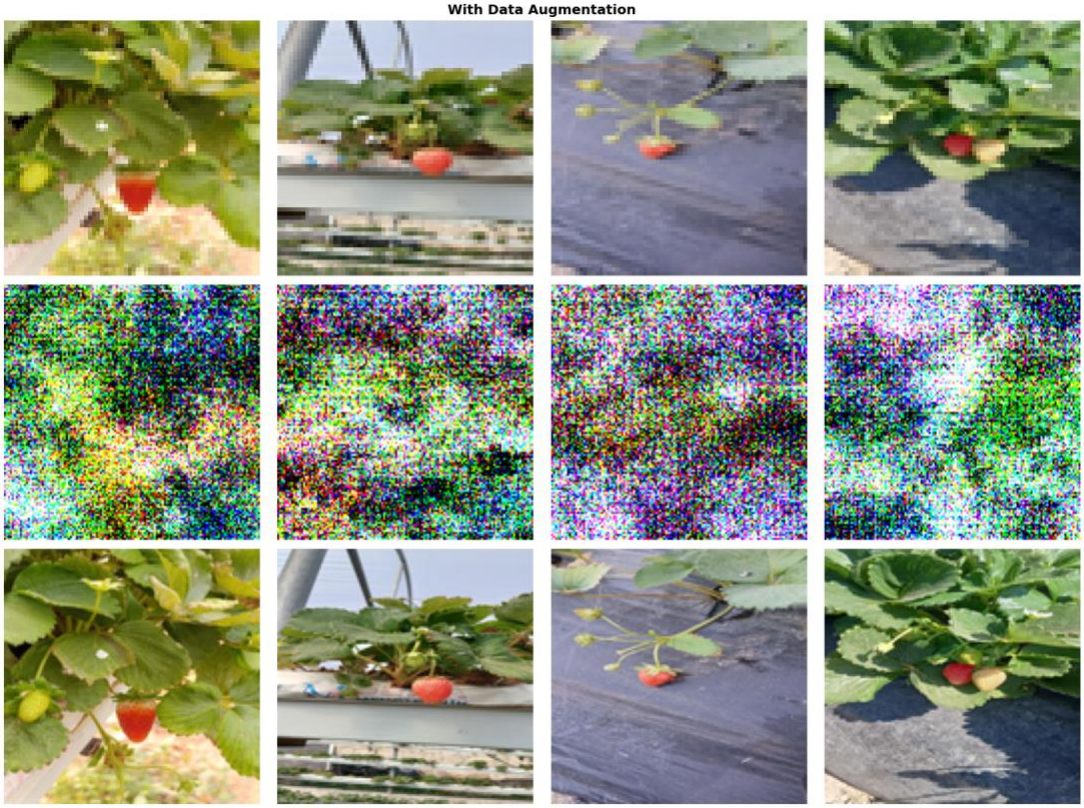


Test Set Performance Comparison

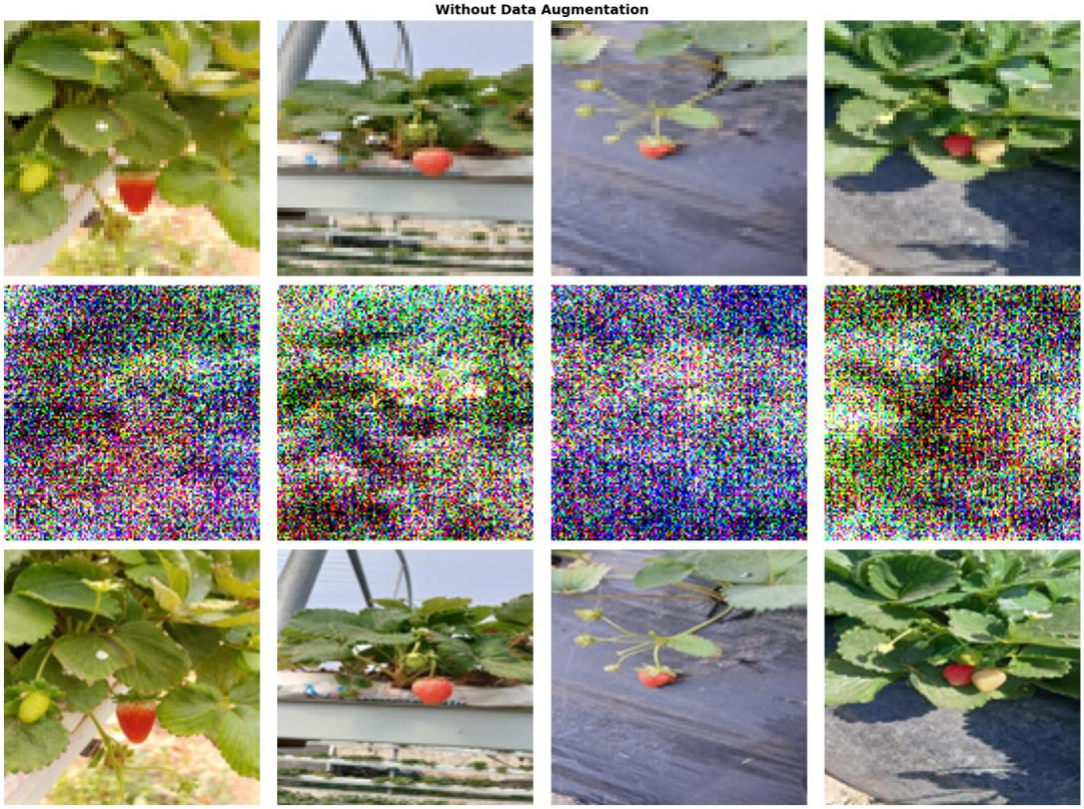
Metric	With Augmentation	Without Augmentation
PSNR (dB)	6.59	6.42
SSIM	0.0102	0.0066
Best Val Loss	0.0382	0.0458

The PSNR and SSIM values are well below typical super-resolution benchmarks. The visual outputs showed mostly noise with vague color patterns corresponding to the original image content. The model began learning the denoising task, as evidenced by the decreasing training loss, but did not converge enough to produce recognizable outputs.

With Data Augmentation

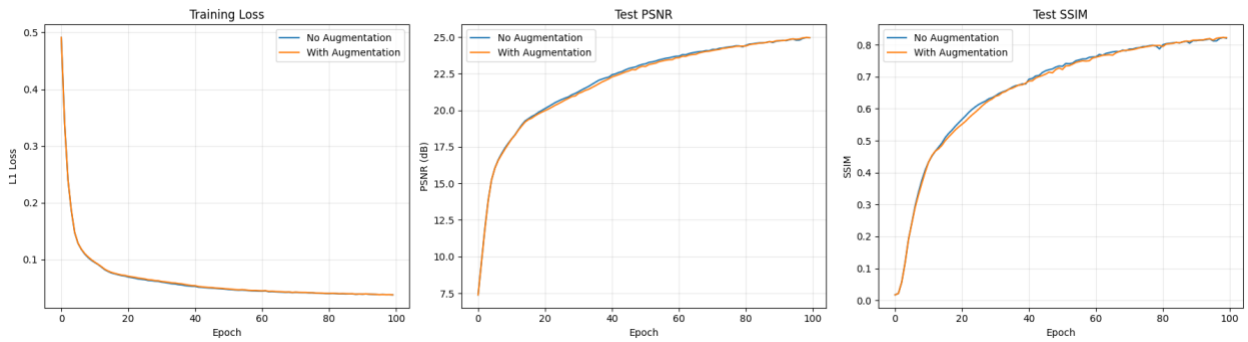


Without Data Augmentation



5.2 Direct Prediction Results

The direct prediction model showed strong improvement over the course of training. PSNR increased steadily from about 6 dB in early epochs to approximately 25 dB by epoch 100. SSIM followed a similar trajectory, reaching above 0.82 by the end of training.



Direct Prediction: Test Set Performance

Metric	With Augmentation	Without Augmentation
PSNR (dB)	24.96	24.95
SSIM	0.8224	0.8206

The visual outputs were substantially better than the diffusion model. The super-resolved images showed clear strawberry shapes, recognizable leaf textures, and accurate color reproduction. Some fine details like individual seeds and text on labels were smoothed out, which is expected with L1 loss, but the overall image quality was close to the ground truth.

5.3 Augmentation Comparison

Data augmentation had a small but measurable effect in the diffusion model, where the augmented version outperformed on all three metrics (PSNR 6.59 vs 6.42, SSIM 0.0102 vs 0.0066, validation loss 0.0382 vs 0.0458). In the direct prediction model, augmentation had almost no effect, with both versions producing nearly identical PSNR and SSIM scores (24.96 vs 24.95 PSNR, 0.8224 vs 0.8206 SSIM).

This suggests that augmentation provides more benefit when the training task is harder and the model is more prone to overfitting. The diffusion model, with its more complex training objective, benefited from

the additional variety. The direct prediction model, with its simpler task and smaller parameter count, did not overfit enough for augmentation to make a meaningful difference within 100 epochs.

6. Discussion

6.1 Why the Diffusion Approach Struggled

The diffusion model produced poor visual results for several interconnected reasons. First, diffusion models are fundamentally data-hungry. The original DiT was trained on 1.2 million ImageNet images over hundreds of thousands of iterations. Training on 566 images for 100 epochs does not provide nearly enough gradient updates for the model to learn the full denoising trajectory across all 1000 noise levels.

Second, the model operated in pixel space rather than the compressed latent space that the original DiT uses. Working at full 128x128 resolution with 1024 patch tokens means each attention computation is expensive, and the model needs more capacity to learn meaningful representations at the pixel level.

Third, the diffusion inference process is sensitive to training quality. If the model has not learned to denoise accurately at every timestep, errors compound during the 50-step sampling process. A small mistake at an early denoising step gets amplified through all subsequent steps, resulting in the noisy outputs observed.

6.2 Why Direct Prediction Worked Better

The direct prediction model performed significantly better because the task itself is simpler. Instead of learning to predict noise at 1000 different levels and then running a multi-step sampling process at inference, it learns a single mapping from low-resolution to high-resolution. Each training step provides a direct gradient signal about image quality, whereas the diffusion model only gets indirect feedback through noise prediction.

The smaller model size (7.2M vs 34.3M parameters) was also an advantage in this case. With only 566 training images, a smaller model is less prone to overfitting and can generalize better from limited data. The lower token count (256 vs 1024) made training faster, allowing the model to see more effective gradient updates in the same wall-clock time.

The use of L1 loss instead of MSE also contributed. L1 loss penalizes all errors equally regardless of magnitude, while MSE disproportionately penalizes large errors. This means MSE pushes the model toward conservative, blurry predictions that avoid large pixel errors. L1 allows the model to take more risks with sharp edges and fine details.

6.3 When Diffusion Would Be the Better Choice

Despite the poor results here, diffusion-based super-resolution is not inherently worse. With sufficient data (tens of thousands of images), longer training (hundreds of thousands of iterations), operation in latent space via a pretrained VAE, and a larger model, diffusion approaches can produce sharper and more detailed outputs than direct prediction methods. This is because the iterative sampling process can commit to specific fine details at each step rather than being forced to predict everything at once, which avoids the regression-to-the-mean problem where direct prediction models blur out ambiguous regions.

For this specific scenario, with a small dataset and limited compute, direct prediction was clearly the right approach. The diffusion model serves as a useful comparison point that demonstrates why architecture choice needs to be matched to the available resources.

7. Using the Trained Model

After training, the model weights are saved as checkpoint files (with the .pt extension). These files contain all the learned parameter values but not the model architecture itself. To use the trained model on a new image, three things are needed: the architecture code that defines the model structure, the saved weight file, and a short script that loads both and processes the input.

The process involves rebuilding the model from the architecture code, loading the trained weights into it, preparing the input image by resizing it to 64x64 and converting it to a tensor, running a single forward pass through the model, and saving or displaying the 128x128 output. No GPU is strictly required for inference on a single image, though it speeds things up.

In a production setting, this would typically be wrapped in a web API using a framework like Flask or FastAPI. A user would upload an image through a web interface, the server would run the model, and the upscaled result would be returned. This is essentially how commercial upscaling tools from companies like Topaz Labs and the Super Resolution features in Adobe Photoshop work behind the scenes.

For this project, the trained weights and architecture code are sufficient to reproduce the results. The Kaggle notebook saves checkpoints to the output directory, which can be downloaded and loaded into any Python environment with PyTorch installed.

8. Limitations and Future Improvements

The main limitation of this project is the dataset size. With only 566 images, neither model had enough data to reach its full potential, though this affected the diffusion model far more severely. A larger and more diverse dataset would improve both models but would particularly benefit the diffusion approach.

Other limitations include the lack of a learning rate scheduler, which could help both models converge more smoothly in later epochs. The diffusion model would also benefit from operating in latent space

with a pretrained VAE, which would reduce the computational burden and allow the model to focus on learning semantic rather than pixel-level features. Implementing DDIM sampling instead of the standard DDPM sampling could also improve diffusion inference quality by using a more efficient denoising trajectory.

For the direct prediction model, the outputs show some smoothing of fine details, which is a known characteristic of L1 loss. Adding a perceptual loss component (comparing features from a pretrained VGG network rather than raw pixels) or an adversarial loss (training a discriminator to distinguish real from generated images) could push the output quality further toward photorealistic results.

9. Conclusion

This project demonstrated two ways to remodel the Diffusion Transformer architecture for image super-resolution. The diffusion-based approach preserved the iterative denoising process but struggled with limited data, producing noisy outputs with PSNR around 6.5 dB. The direct prediction approach removed the diffusion process and achieved strong results with PSNR around 25 dB and SSIM above 0.82, producing visually recognizable outputs that closely matched the ground truth.

The augmentation comparison showed that data augmentation had a small positive effect on the diffusion model but negligible impact on the direct prediction model, suggesting that augmentation matters more when the training task is complex and the model is more likely to overfit.

The central takeaway is that choosing the right training method matters as much as choosing the right architecture. The same DiT transformer blocks produced vastly different results depending on whether they were used for iterative diffusion or direct prediction, and this difference was amplified by the constraints of a small dataset.

10. References

Peebles, W., and Xie, S. (2023). Scalable Diffusion Models with Transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 4195-4205.*

Ho, J., Jain, A., and Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *Advances in Neural Information Processing Systems, 33, 6840-6851.*

Nichol, A. Q., and Dhariwal, P. (2021). Improved Denoising Diffusion Probabilistic Models. *Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 8162-8171.*

Dosovitskiy, A., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *International Conference on Learning Representations (ICLR).*

Shi, W., et al. (2016). Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1874-1883.*

Loshchilov, I., and Hutter, F. (2019). Decoupled Weight Decay Regularization. *International Conference on Learning Representations (ICLR).*

Yurdakul, M., Bastug, Z. S., Gok, A. E., and Tasdemir, S. A Novel Public Dataset for Strawberry Ripeness Detection and Comparative Evaluation of YOLO-Based Models.